

Assignment

PDEs Implementation task in 2D

by

Agenorwoth Samuel



**KARLSTAD
UNIVERSITY**

February 6, 2024

Introduction

In this implementation task, we look at how to assembly the matrix and later write a code for the specific task at hand. We then produce the plots for the solution.

On the other-hand, we use FEniCS project and present the plots of the solution as well as a bonus on how easier it is as compared to the manual assembling of the matrix.

Lastly, I suggest a recommendation for the next coming students especially on this specific implementation task.

Question

Numerically solve the equation

$$\begin{aligned} u - \nabla \cdot \nabla u &= 1 \text{ in } \Omega \\ u &= 0 \text{ on } \partial\Omega \end{aligned} \quad (1)$$

(a) using the standard first-order Lagrangian basis functions when Ω is the unit square.

We carry out the transformation for $P_i(x_i, y_i)$ for $i = 1, 2, 3$ to a standard triangle $(0, 0) (1, 0) (0, 1)$. We have the diagrammatic presentation with $\bar{x}_i = (x_i, y_i)$ as

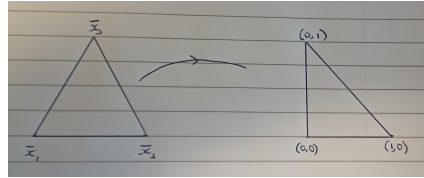


Figure 1: Transformation to standard triangle

We define our matrix as

$$B(x, y) = \begin{pmatrix} a & b \\ a' & b' \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} c \\ c' \end{pmatrix} \quad (2)$$

We carry out transformation at each point $P_i(x_i, y_i)$ for $i = 1, 2, 3$ and aim to find the values

of a, b, c, a', b', c' as

$$ax_1 + by_1 + c = 0 \quad (3)$$

$$a'x_1 + b'y_1 + c' = 0 \quad (4)$$

$$ax_2 + by_2 + c = 1 \quad (5)$$

$$a'x_2 + b'y_2 + c' = 0 \quad (6)$$

$$ax_3 + by_3 + c = 0 \quad (7)$$

$$a'x_3 + b'y_3 + c' = 1 \quad (8)$$

We write (3)–(8) in a matrix form as

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \\ x_3 & y_3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_3 & y_3 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ a' \\ b' \\ c' \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (9)$$

We write its python code and use **scipy library** to solve for the values a, b, c, a', b', c' , We write the code to return (2) as shown

```

1  def Btransf(p1,p2,p3):
2      A=np.zeros((6,6))
3      Lef=np.array([0,0,1,0,0,1])
4      A[:,2]=np.array([1,0,1,0,1,0])
5      A[:,5]=np.array([0,1,0,1,0,1])
6      A[:,0]=np.array([p1[0],0,p2[0],0,p3[0],0])
7      A[:,1]=np.array([p1[1],0,p2[1],0,p3[1],0])
8      A[:,3]=np.array([0,p1[0],0,p2[0],0,p3[0]])
9      A[:,4]=np.array([0,p1[1],0,p2[1],0,p3[1]])
10     B_standard=scp.solve(A, Lef)
11     # The matrix or linear part
12     B_tr=np.array([[B_standard[0],B_standard[1]],[B_standard
13     [3],B_standard[4]]])
14     # The intercepts
15     vect=np.array([B_standard[2],B_standard[5]])
16     return B_tr,vect
17     # Collecting the matrix
18 def proj1Btransf(p1,p2,p3):
19     return Btransf(p1,p2,p3)[0]
20     # Collecting the vector
21 def proj2Btransf(p1,p2,p3):
22     return Btransf(p1,p2,p3)[1]
```

We draw our square grid and label the elements $E_1, E_2, \dots$

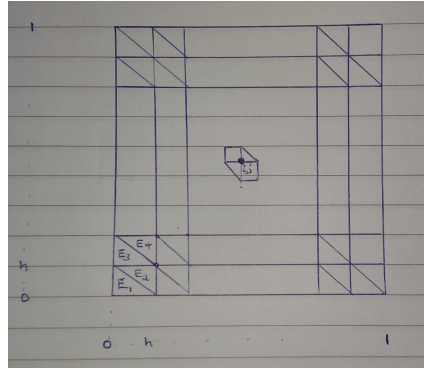


Figure 2: Square grid

At each node, we shall have a hat function and each hat function will contain 6 elements. We shall locate the node at position (i_k, j_k) where the position of the number k is within

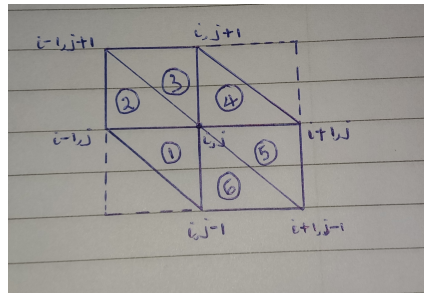


Figure 3: Representation of the hat at node i, j

a conceptual grid by determining the row (i_k) and column (j_k) indices where k would be positioned if the numbers $1, 2, \dots, N$ were laid out in the grid row by row. We write the code for locating the hat function as

```

1 # so that one can know the nodes to use and the elements to
  consider.
2 def i_k(k,N):
3     ik=min([i for i in range(1,N+1) if k<=i*N])
4     jk=k-(ik-1)*N
5     ik=ik-1
6     jk=jk-1
7     return np.array([ik,jk])

```

We keep in mind that we shall have some overlapping of the hat functions within its neighborhood in the grid. From here, the variational form of $u - \nabla \cdot \nabla u = 1$ in Ω and $u = 0$

on $\partial\Omega$ is given by

$$\int_{\Omega} u\psi_k + \int_{\Omega} \nabla u \nabla \psi_k = \int_{\Omega} \psi_k \quad (10)$$

From (10), we proceed with $u = a_{\ell}\psi_{\ell}$ as

$$\begin{aligned} \int_{\Omega} u\psi_k &= \int_{\Omega} a_{k-1}\psi_{k-1}\psi_k + \int_{\Omega} a_{k-N}\psi_{k-N}\psi_k + \int_{\Omega} a_{k-N+1}\psi_{k-N+1}\psi_k + \int_{\Omega} a_{k+1}\psi_{k+1}\psi_k \\ &+ \int_{\Omega} a_{k+N}\psi_{k+N}\psi_k + \int_{\Omega} a_{k+N-1}\psi_{k+N-1}\psi_k \end{aligned} \quad (11)$$

We write (11) as

$$\begin{aligned} \int_{\Omega} u\psi_k &= a_{k-1} \left[\int_{E_{6,k}} \hat{\psi}_{1,E_{4,k-1}} \hat{\psi}_{3,E_{6,k}} + \int_{E_{1,k}} \hat{\psi}_{1,E_{3,k-1}} \hat{\psi}_{3,E_{1,k}} \right] \\ &+ a_{k-N} \left[\int_{E_{1,k}} \hat{\psi}_{2,E_{5,k-N}} \hat{\psi}_{3,E_{1,k}} + \int_{E_{2,k}} \hat{\psi}_{1,E_{4,k-N}} \hat{\psi}_{2,E_{2,k}} \right] \\ &+ a_{k-N+1} \left[\int_{E_{2,k}} \hat{\psi}_{3,E_{6,k-N+1}} \hat{\psi}_{2,E_{2,k}} + \int_{E_{3,k}} \hat{\psi}_{2,E_{5,k-N+1}} \hat{\psi}_{1,E_{3,k}} \right] \\ &+ a_{k+1} \left[\int_{E_{3,k}} \hat{\psi}_{3,E_{1,k+1}} \hat{\psi}_{1,E_{3,k}} + \int_{E_{4,k}} \hat{\psi}_{3,E_{6,k+1}} \hat{\psi}_{1,E_{4,k}} \right] \\ &+ a_{k+N} \left[\int_{E_{4,k}} \hat{\psi}_{2,E_{2,k+N}} \hat{\psi}_{1,E_{4,k}} + \int_{E_{5,k}} \hat{\psi}_{3,E_{1,k+N}} \hat{\psi}_{2,E_{5,k}} \right] \\ &+ a_{k+N-1} \left[\int_{E_{5,k}} \hat{\psi}_{1,E_{3,k+N-1}} \hat{\psi}_{2,E_{5,k}} + \int_{E_{6,k}} \hat{\psi}_{2,E_{2,k+N-1}} \hat{\psi}_{3,E_{6,k}} \right] \end{aligned} \quad (12)$$

We transform E to Δ in (12) through Jacobi by letting $\xi = T^{-1}(\xi^*, \eta^*)$ and $\eta = T^{-1}(\xi^*, \eta^*)$ hence $\Rightarrow d\xi d\eta = |J_{T^{-1}}| d\xi^* d\eta^*$. We now compute the Jacobi from by first re-writing our transformation as

$$\begin{aligned} T = \begin{pmatrix} a\xi + b\eta + c \\ a'\xi + b'\eta + c' \end{pmatrix} \text{ then } |J_T| &= \left| \begin{bmatrix} a & b \\ a' & b' \end{bmatrix} \right| \\ |J_T| &= |ab' - a'b| \end{aligned} \quad (13)$$

where we compute a, b, a', b' using the first code written. We have

$$\begin{aligned}
\int_{\Omega} u \psi_k &= \frac{1}{|J_T|} a_{k-1} \left[\int_{\Delta} \hat{\psi}_{1,E_{4,k-1}} \hat{\psi}_{3,E_{6,k}} + \int_{\Delta} \hat{\psi}_{1,E_{3,k-1}} \hat{\psi}_{3,E_{1,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k-N} \left[\int_{\Delta} \hat{\psi}_{2,E_{5,k-N}} \hat{\psi}_{3,E_{1,k}} + \int_{\Delta} \hat{\psi}_{1,E_{4,k-N}} \hat{\psi}_{2,E_{2,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k-N+1} \left[\int_{\Delta} \hat{\psi}_{3,E_{6,k-N+1}} \hat{\psi}_{2,E_{2,k}} + \int_{\Delta} \hat{\psi}_{2,E_{5,k-N+1}} \hat{\psi}_{1,E_{3,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k+1} \left[\int_{\Delta} \hat{\psi}_{3,E_{1,k+1}} \hat{\psi}_{1,E_{3,k}} + \int_{\Delta} \hat{\psi}_{3,E_{6,k+1}} \hat{\psi}_{1,E_{4,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k+N} \left[\int_{\Delta} \hat{\psi}_{2,E_{2,k+N}} \hat{\psi}_{1,E_{4,k}} + \int_{\Delta} \hat{\psi}_{3,E_{1,k+N}} \hat{\psi}_{2,E_{5,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k+N-1} \left[\int_{\Delta} \hat{\psi}_{1,E_{3,k+N-1}} \hat{\psi}_{2,E_{5,k}} + \int_{\Delta} \hat{\psi}_{2,E_{2,k+N-1}} \hat{\psi}_{3,E_{6,k}} \right]
\end{aligned} \tag{14}$$

We apply the same procedures for $\int_{\Omega} \nabla u \nabla \psi_k$ thus obtaining

$$\begin{aligned}
\int_{\Omega} \nabla u \nabla \psi_k &= \frac{1}{|J_T|} a_{k-1} \left[\int_{\Delta} \nabla \hat{\psi}_{1,E_{4,k-1}} \nabla \hat{\psi}_{3,E_{6,k}} + \int_{\Delta} \nabla \hat{\psi}_{1,E_{3,k-1}} \nabla \hat{\psi}_{3,E_{1,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k-N} \left[\int_{\Delta} \nabla \hat{\psi}_{2,E_{5,k-N}} \nabla \hat{\psi}_{3,E_{1,k}} + \int_{\Delta} \nabla \hat{\psi}_{1,E_{4,k-N}} \nabla \hat{\psi}_{2,E_{2,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k-N+1} \left[\int_{\Delta} \nabla \hat{\psi}_{3,E_{6,k-N+1}} \nabla \hat{\psi}_{2,E_{2,k}} + \int_{\Delta} \nabla \hat{\psi}_{2,E_{5,k-N+1}} \nabla \hat{\psi}_{1,E_{3,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k+1} \left[\int_{\Delta} \nabla \hat{\psi}_{3,E_{1,k+1}} \nabla \hat{\psi}_{1,E_{3,k}} + \int_{\Delta} \nabla \hat{\psi}_{3,E_{6,k+1}} \nabla \hat{\psi}_{1,E_{4,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k+N} \left[\int_{\Delta} \nabla \hat{\psi}_{2,E_{2,k+N}} \nabla \hat{\psi}_{1,E_{4,k}} + \int_{\Delta} \nabla \hat{\psi}_{3,E_{1,k+N}} \nabla \hat{\psi}_{2,E_{5,k}} \right] \\
&+ \frac{1}{|J_T|} a_{k+N-1} \left[\int_{\Delta} \nabla \hat{\psi}_{1,E_{3,k+N-1}} \nabla \hat{\psi}_{2,E_{5,k}} + \int_{\Delta} \nabla \hat{\psi}_{2,E_{2,k+N-1}} \nabla \hat{\psi}_{3,E_{6,k}} \right]
\end{aligned} \tag{15}$$

and Let $b_k = \int_{\Omega} \psi_k$, then we can write b_k as

$$b_k = \int_{E_{1,l}} \hat{\psi}_{3,E_{1,l}} + \int_{E_{2,l}} \hat{\psi}_{2,E_{2,l}} + \int_{E_{3,l}} \hat{\psi}_{1,E_{3,l}} + \int_{E_{4,l}} \hat{\psi}_{1,E_{4,l}} + \int_{E_{5,l}} \hat{\psi}_{2,E_{5,l}} + \int_{E_{6,l}} \hat{\psi}_{3,E_{6,l}} \tag{16}$$

Similarly, the transformation from E to Δ in (16) through Jacobi as

$$\begin{aligned}
b_k &= \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{3,E_{1,l}} + \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{2,E_{2,l}} + \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{1,E_{3,l}} + \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{1,E_{4,l}} + \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{2,E_{5,l}} + \frac{1}{h^2} \int_{\Delta} \hat{\psi}_{3,E_{6,l}} \\
&= \frac{1}{|J_T|} \int_{\Delta} \left(\hat{\psi}_{3,E_{1,l}} + \hat{\psi}_{2,E_{2,l}} + \hat{\psi}_{1,E_{3,l}} + \hat{\psi}_{1,E_{4,l}} + \hat{\psi}_{2,E_{5,l}} + \hat{\psi}_{3,E_{6,l}} \right)
\end{aligned}$$

For each part of b_k we obtain the same value for the $\hat{\psi}$ with respect to the element as

$$\begin{aligned}\frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{1,E} &= \frac{1}{|J_T|} \int_0^1 \int_0^{1-x} (1-x-y) dx dy = \frac{1}{6|J_T|} \\ \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{2,E} &= \frac{1}{|J_T|} \int_0^1 \int_0^{1-x} x dx dy = \frac{1}{6|J_T|} \\ \frac{1}{|J_T|} \int_{\Delta} \hat{\psi}_{3,E} &= \frac{1}{|J_T|} \int_0^1 \int_0^{1-x} y dx dy = \frac{1}{6|J_T|}\end{aligned}$$

$$\therefore b_k = \frac{1}{|J_T|} \left(\frac{1}{6} \times 6 \right) = \frac{1}{|J_T|}$$

We now write the code for the hat function built on the nodes of the grid points

```

1      ## The standard hat function
2  def standHat(x,y,i):
3      if i==1:
4          z=1-x-y
5      elif i==2:
6          z=x
7      else:
8          z=y
9      return z
10
11  ## The hat function built upon nodes of the grid points
12  def HatFunck(x,y,k,N):
13      # x,y are point coordinates
14      # k is the hat function's number
15      # the numbers of internal nodes considered
16      h=1./(N+1) # To get N internal nodes
17      ind1=i_k(k,N)
18      ik=ind1[0]+1
19      jk=ind1[1]+1
20      # $(i_k,i_k)$ represent the position of $\text{vfiz}_k$ in the
21      grid points
22      ik0,jk0=ik, jk-1
23      ik2,jk2=ik, jk+1
24      ik3,jk3=ik-1,jk
25      ik4,jk4=ik+1,jk
26      if x==ik*h and y==jk*h:
27          z=1
28          grad=0
29      elif x>ik3*h and x<=ik*h:
30          if y>-x+(ik0+jk0)*h and y<=jk*h:
31              P1=np.array([ik*h,jk*h])
32              P2=np.array([ik0*h,jk0*h])
33              P3=np.array([ik3*h,jk3*h])

```

```

32         transf=Btransf(P2, P3, P1)
33         transfM=proj1Btransf(P2, P3, P1)
34         transfV=proj2Btransf(P2, P3, P1)
35         vec=np.array([x,y])
36         Xtra=np.dot(transfM,vec)+transfV
37         z=standHat(Xtra[0], Xtra[1],3)
38         grad=transfM[1]
39     elif y<=-x+(ik+jk)*h and y>jk*h:
40         P1=np.array([ik*h,jk*h])
41         P2=np.array([ik3*h,jk3*h])
42         P3=np.array([ik3*h,jk2*h])
43         transfM=proj1Btransf(P2, P1, P3)
44         transfV=proj2Btransf(P2, P1, P3)
45         vec=np.array([x,y])
46         Xtra=np.dot(transfM,vec)+transfV
47         z=standHat(Xtra[0], Xtra[1],2)
48         grad=transfM[0]
49     elif y>-x+(ik+jk)*h and y<jk2*h:
50         P1=np.array([ik*h,jk*h])
51         P2=np.array([ik*h,jk2*h])
52         P3=np.array([ik3*h,jk2*h])
53         transfM=proj1Btransf(P1, P3, P2)
54         transfV=proj2Btransf(P1, P3, P2)
55         vec=np.array([x,y])
56         Xtra=np.dot(transfM,vec)+transfV
57         z=standHat(Xtra[0], Xtra[1],1)
58         grad=-transfM[0]-transfM[1]
59     else:
60         z=0
61         grad=np.zeros(2)
62     elif x>ik*h and x<ik4*h:
63         if y<-x+(ik4+jk4)*h and y>=jk*h:
64             P1=np.array([ik*h,jk*h])
65             P2=np.array([ik2*h,jk2*h])
66             P3=np.array([ik4*h,jk4*h])
67             transf=Btransf(P2, P3, P1)
68             transfM=proj1Btransf(P1, P3, P2)
69             transfV=proj2Btransf(P1, P3, P2)
70             vec=np.array([x,y])
71             Xtra=np.dot(transfM,vec)+transfV
72             z=standHat(Xtra[0], Xtra[1],1)
73             grad=-transfM[1]-transfM[0]
74     elif y>=-x+(ik4+jk0)*h and y<jk*h:

```

```

75     P1=np.array([ik*h,jk*h])
76     P2=np.array([ik4*h,jk0*h])
77     P3=np.array([ik4*h,jk4*h])
78     transfM=proj1Btransf(P2, P1, P3)
79     transfV=proj2Btransf(P2, P1, P3)
80     vec=np.array([x,y])
81     Xtra=np.dot(transfM,vec)+transfV
82     z=standHat(Xtra[0], Xtra[1],2)
83     grad=transfM[0]
84     elif y<-x+(ik4+jk0)*h and y>jk0*h:
85         P1=np.array([ik*h,jk*h])
86         P2=np.array([ik4*h,jk0*h])
87         P3=np.array([ik0*h,jk0*h])
88         transfM=proj1Btransf(P3, P2, P1)
89         transfV=proj2Btransf(P3, P2, P1)
90         vec=np.array([x,y])
91         Xtra=np.dot(transfM,vec)+transfV
92         z=standHat(Xtra[0], Xtra[1],3)
93         grad=transfM[1]
94     else:
95         z=0
96         grad=np.zeros(2)
97     else:
98         z=0
99         grad=np.array([0,0])
100     return z,grad # we also return the gradient of the $\vfi_k$

```

We write the code to compute the gradient with respect to the element as well. It will be attached in the appendix for the general code due to its length.

For the remaining part of the code, we have an initial setup for variables $x_0 = 0$ and $x_{\text{fin}} = 1$ with N representing the number of divisions in the domain and step size $h_0 = \frac{1}{N+1}$ with $\text{dim0} = N^2$ (grid of points in a square domain).

We further define I , J , S , and B as initialized as zero arrays where I and J are respectively the row and column indices for a sparse matrix, S are the values corresponding to the I , J indices in the sparse matrix, and B is the right-hand side vector in a linear system of equations

```

1  x0=0
2  xfin=1
3  N=15
4  h0=1./(N+1)
5  dim0=N**2
6  if N==1:

```

```
7     dim=1
8 elif N==2:
9     dim=14
10 else:
11     dim=6+20*(N-2)+8+7*((N-2)**2)
12 I=np.zeros(dim)
13 J=np.zeros(dim)
14 S=np.zeros(dim)
15 B=np.zeros(dim0)
```

We write an if loop to locate the position of the indices in I, J, S arrays based on the input k and the grid size N where for $N = 1$ and $N = 2$, we have specific positions are returned.

For $N > 2$, the function handles the first and last grid points ($k = 1$ and $k = N$) and we use `np.concatenate` to build up the `positions` array which suggests that each grid point could be associated with multiple indices in the I, J, S arrays.

We write the additional lines of code perform the assembly of a sparse matrix and solve the corresponding linear system

We represent the written final code(commented code) in the Appendix and represent the plots with the corresponding grid number as shown below

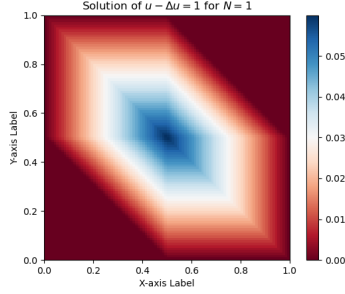


Figure 4

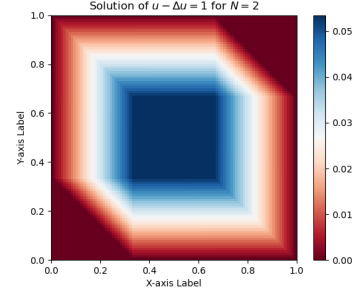


Figure 5

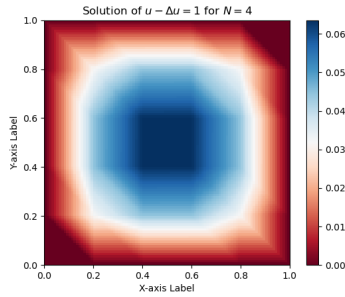


Figure 6

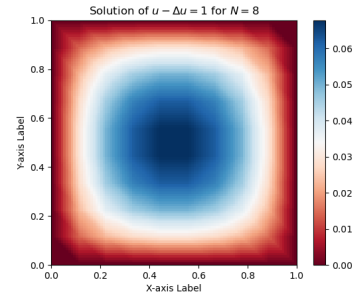


Figure 7

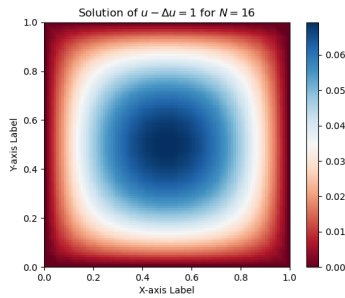


Figure 8

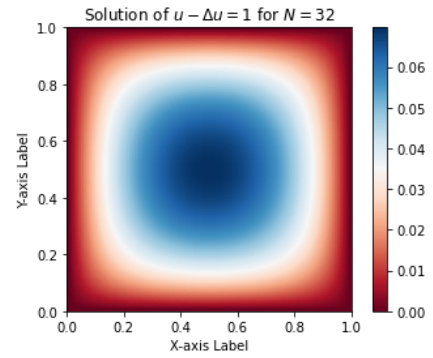


Figure 9

(b) using the standard first-order Lagrangian basis functions when Ω is L-Shaped.

Here, we use the same code but only adjust it to remove the right corner and provide the plots as shown. The code is attached in the Appendix as well

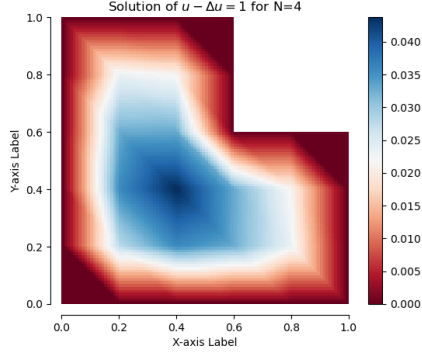


Figure 10

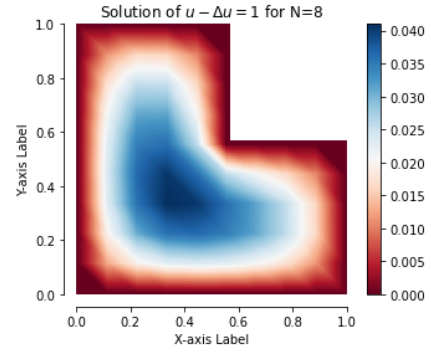


Figure 11

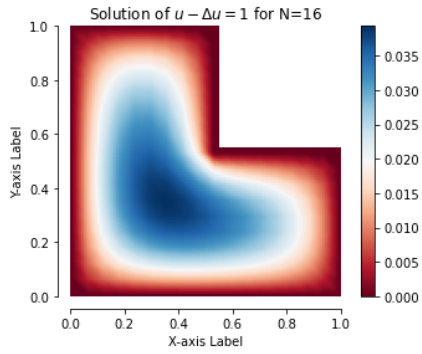


Figure 12

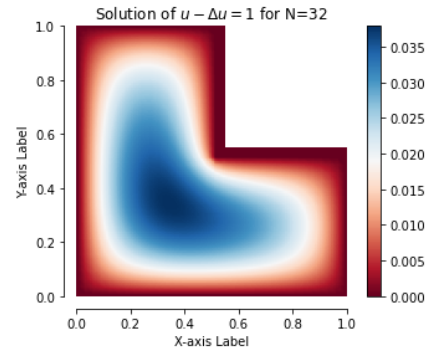


Figure 13

The above work is for the manual setup of the code. We now briefly look at FEniCS library implementation and provide its plots in the next page.

Using FEniCS Project

Using the FEniCS project, we proceed by first writing the weak form of

$$\begin{aligned} u - \nabla \cdot \nabla u &= 1 \text{ in } \Omega \\ u &= 0 \text{ on } \partial\Omega \end{aligned} \quad (17)$$

Let $v \in H_0^1(\Omega)$ be the test function, then we multiply (17) by v and apply boundary conditions as follows

$$\begin{aligned} \int_{\Omega} u \cdot v \, dx - \int_{\Omega} \nabla(\nabla u) \cdot v \, dx &= \int_{\Omega} 1 \cdot v \, dx \\ \int_{\Omega} u \cdot v \, dx + \int_{\Omega} \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} \nabla u \cdot v \cdot n \, ds &= \int_{\Omega} v \, dx \\ \int_{\Omega} u \cdot v \, dx + \int_{\Omega} \nabla u \cdot \nabla v \, dx &= \int_{\Omega} v \, dx \end{aligned} \quad (18)$$

Then (18) is the weak form of (17). We then write a code including the weak form. The plots where Ω is a unit square is produced with varying meshes as

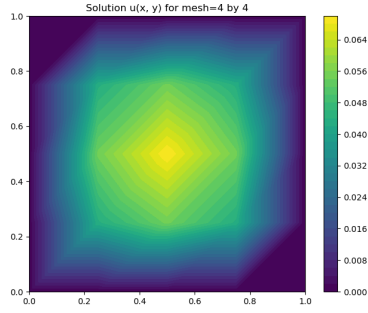


Figure 14: Mesh size 4 by 4

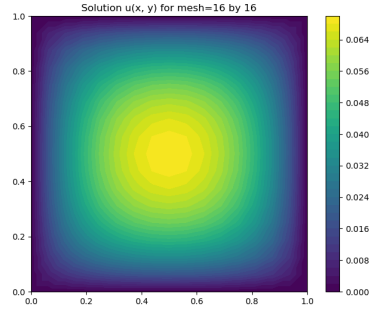


Figure 15: Mesh size 16 by 16

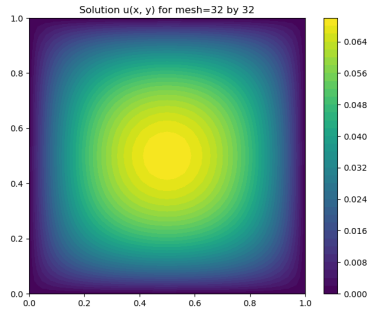


Figure 16: Mesh size 32 by 32

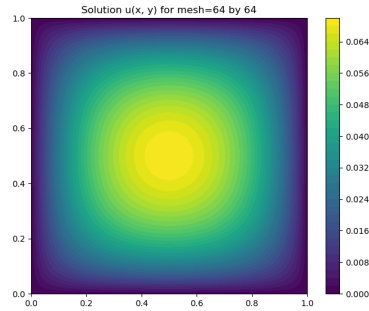


Figure 17: Mesh size 64 by 64

The plots above show how the solution looks on varying the mesh.

On the other-hand, we produce the plots where Ω is L-shaped with the upper right corner of a unit square removed.

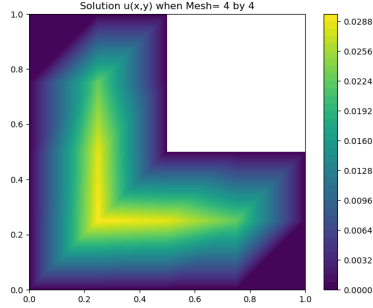


Figure 18: Mesh size 4 by 4

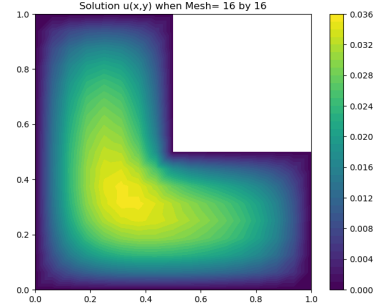


Figure 19: Mesh size 16 by 16

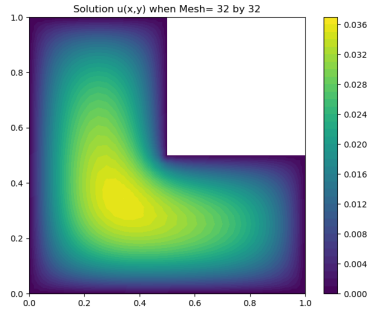


Figure 20: Mesh size 32 by 32

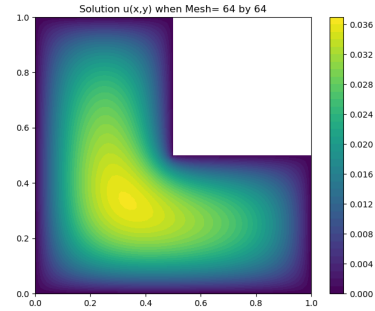


Figure 21: Mesh size 64 by 64

The plot below shows how the error changes with the Mesh

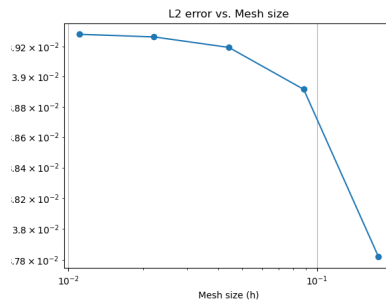


Figure 22: Mesh Vs error

We attach the code in the Appendix for Ω as the unit square and Ω as the L-shaped (remove upper right corner of unit square) corresponding to both the manual setup and FEniCS codes.

February 6, 2024

Recommendation for the upcoming students

I recommend the task for this implementation in 2D to be handled in a group of 2 due to its nature.

Appendix for Manual Set Up

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import sympy as sp
4 import scipy.linalg as scp
5 from scipy.sparse import csr_matrix
6 from mpl_toolkits.mplot3d import Axes3D
7 from matplotlib import cm
8 from matplotlib.ticker import LinearLocator
9 from pylab import meshgrid,cm,imshow,contour,clabel,colorbar,
    axis,title,show
10
11 ## Transformation to standard triangle
12 # Function to produce the affine transformation from a given
    triangle  $P_i(x_i,y_i)$ ,  $i=1,2,3$ ,
13 # the standard triangle  $A(0,0)$ ,  $B(1,0)$ ,  $C(0,1)$ .
14 def Btransf(p1,p2,p3):
15     A=np.zeros((6,6))
16     Lef=np.array([0,0,1,0,0,1])
17     A[:,2]=np.array([1,0,1,0,1,0])
18     A[:,5]=np.array([0,1,0,1,0,1])
19     A[:,0]=np.array([p1[0],0,p2[0],0,p3[0],0])
20     A[:,1]=np.array([p1[1],0,p2[1],0,p3[1],0])
21     A[:,3]=np.array([0,p1[0],0,p2[0],0,p3[0]])
22     A[:,4]=np.array([0,p1[1],0,p2[1],0,p3[1]])
23     B_standard=scp.solve(A, Lef)
24     # The matrix or linear part
25     B_tr=np.array([[B_standard[0],B_standard[1]],[B_standard
    [3],B_standard[4]]])
26     # The intercepts
27     vect=np.array([B_standard[2],B_standard[5]])
28     return B_tr,vect
29 # Collecting the matrix
30 def proj1Btransf(p1,p2,p3):
31     return Btransf(p1,p2,p3)[0]
32 # Collecting the vector
33 def proj2Btransf(p1,p2,p3):
34     return Btransf(p1,p2,p3)[1]
35 # Bijection from  $\{1,2,\dots,N^2\}$  to  $\{(i,j):1\leq i,j\leq N\}$ 
36 # This function helps to locate the hat function for the grid
    points
```

```

37 # so that one can know the nodes to use and the elements to
    consider.
38 def i_k(k,N):
39     ik=min([i for i in range(1,N+1) if k<=i*N])
40     jk=k-(ik-1)*N
41     ik=ik-1
42     jk=jk-1
43     return np.array([ik,jk])
44 ## The standard hat function
45 def standHat(x,y,i):
46     if i==1:
47         z=1-x-y
48     elif i==2:
49         z=x
50     else:
51         z=y
52     return z
53 ## The hat function built upon nodes of the grid points
54 def HatFunck(x,y,k,N):
55     # x,y are point coordinates
56     # k is the hat function's number
57     # the numbers of internal nodes considered
58     h=1./(N+1) # To get N internal nodes
59     ind1=i_k(k,N)
60     ik=ind1[0]+1
61     jk=ind1[1]+1
62     # $(i_k,i_k)$ represent the position of $\text{vfiz}_k$ in the
    grid points
63     ik0,jk0=ik, jk-1
64     ik2,jk2=ik, jk+1
65     ik3,jk3=ik-1,jk
66     ik4,jk4=ik+1,jk
67     if x==ik*h and y==jk*h:
68         z=1
69         grad=0
70     elif x>ik3*h and x<=ik*h:
71         if y>-x+(ik0+jk0)*h and y<=jk*h:
72             P1=np.array([ik*h,jk*h])
73             P2=np.array([ik0*h,jk0*h])
74             P3=np.array([ik3*h,jk3*h])
75             transf=Btransf(P2, P3, P1)
76             transfM=proj1Btransf(P2, P3, P1)
77             transfV=proj2Btransf(P2, P3, P1)

```

```

78         vec=np.array([x,y])
79         Xtra=np.dot(transfM,vec)+transfV
80         z=standHat(Xtra[0], Xtra[1],3)
81         grad=transfM[1]
82     elif y<=-x+(ik+jk)*h and y>jk*h:
83         P1=np.array([ik*h,jk*h])
84         P2=np.array([ik3*h,jk3*h])
85         P3=np.array([ik3*h,jk2*h])
86         transfM=proj1Btransf(P2, P1, P3)
87         transfV=proj2Btransf(P2, P1, P3)
88         vec=np.array([x,y])
89         Xtra=np.dot(transfM,vec)+transfV
90         z=standHat(Xtra[0], Xtra[1],2)
91         grad=transfM[0]
92     elif y>-x+(ik+jk)*h and y<jk2*h:
93         P1=np.array([ik*h,jk*h])
94         P2=np.array([ik*h,jk2*h])
95         P3=np.array([ik3*h,jk2*h])
96         transfM=proj1Btransf(P1, P3, P2)
97         transfV=proj2Btransf(P1, P3, P2)
98         vec=np.array([x,y])
99         Xtra=np.dot(transfM,vec)+transfV
100        z=standHat(Xtra[0], Xtra[1],1)
101        grad=-transfM[0]-transfM[1]
102    else:
103        z=0
104        grad=np.zeros(2)
105    elif x>ik*h and x<ik4*h:
106        if y<-x+(ik4+jk4)*h and y>=jk*h:
107            P1=np.array([ik*h,jk*h])
108            P2=np.array([ik2*h,jk2*h])
109            P3=np.array([ik4*h,jk4*h])
110            transf=Btransf(P2, P3, P1)
111            transfM=proj1Btransf(P1, P3, P2)
112            transfV=proj2Btransf(P1, P3, P2)
113            vec=np.array([x,y])
114            Xtra=np.dot(transfM,vec)+transfV
115            z=standHat(Xtra[0], Xtra[1],1)
116            grad=-transfM[1]-transfM[0]
117        elif y>=-x+(ik4+jk0)*h and y<jk*h:
118            P1=np.array([ik*h,jk*h])
119            P2=np.array([ik4*h,jk0*h])
120            P3=np.array([ik4*h,jk4*h])

```

```

121         transfM=proj1Btransf(P2, P1, P3)
122         transfV=proj2Btransf(P2, P1, P3)
123         vec=np.array([x,y])
124         Xtra=np.dot(transfM,vec)+transfV
125         z=standHat(Xtra[0], Xtra[1],2)
126         grad=transfM[0]
127         elif y<-x+(ik4+jk0)*h and y>jk0*h:
128             P1=np.array([ik*h,jk*h])
129             P2=np.array([ik4*h,jk0*h])
130             P3=np.array([ik0*h,jk0*h])
131             transfM=proj1Btransf(P3, P2, P1)
132             transfV=proj2Btransf(P3, P2, P1)
133             vec=np.array([x,y])
134             Xtra=np.dot(transfM,vec)+transfV
135             z=standHat(Xtra[0], Xtra[1],3)
136             grad=transfM[1]
137         else:
138             z=0
139             grad=np.zeros(2)
140     else:
141         z=0
142         grad=np.array([0,0])
143     return z,grad # we also return the gradient of the  $\nabla v_{fi,k}$ 
144 def proj1Hat(x,y,k,N): # To get the hat function
145     return HatFunck(x, y, k, N)[0]
146 def proj2Hat(x,y,k,N): # To get the gradient of the hat
147     function
148     return HatFunck(x, y, k, N)[1]
149 ## Compute the norm squared
150 def norm_2(a):
151     return np.linalg.norm(a)**2
152 ## Computation of the scalar product of the gradient of hat
153     function
154 #  $\nabla v_{fi,k} \cdot \nabla v_{fi,l}$  to used in  $\int_0^1 \nabla v_{fi,k} \cdot \nabla v_{fi,l}$ 
155 def gradcomp(k,l,N):
156     h=1./(N+1)
157     ind=i_k(k,N)
158     ik=ind[0]+1
159     jk=ind[1]+1
160     # building points for computing successfully the gradient
161     if l!=k:
162         if l==k-1:

```

```

161         x1,y1=ik*h+(h/10),(jk-1)*h+(h/10)
162         x2,y2=ik*h-(h/10),jk*h-(h/10)
163     elif l==k-N:
164         x1,y1=ik*h-(h/10),jk*h-(h/10)
165         x2,y2=(ik-1)*h+(h/10),jk*h+(h/10)
166     elif l==k-N+1:
167         x1,y1=(ik-1)*h+(h/10),jk*h+(h/10)
168         x2,y2=ik*h-(h/10),(jk+1)*h-(h/10)
169     elif l==k+1:
170         x1,y1=ik*h-(h/10),(jk+1)*h-(h/10)
171         x2,y2=ik*h+(h/10),jk*h+(h/10)
172     elif l==k+N:
173         x1,y1=ik*h+(h/10),jk*h+(h/10)
174         x2,y2=(ik+1)*h-(h/10),jk*h-(h/10)
175     elif l==k+N-1:
176         x1,y1=(ik+1)*h-(h/10),jk*h-(h/10)
177         x2,y2=ik*h+(h/10),(jk-1)*h+(h/10)
178     else:
179         x1,y1=(ik+2)*h,(jk+2)*h
180         x2,y2=(ik-2)*h,(jk-2)*h
181     s1=np.dot(proj2Hat(x1, y1, k, N),proj2Hat(x1, y1, l, N)
182 )
183     s2=np.dot(proj2Hat(x2, y2, k, N),proj2Hat(x2, y2, l, N)
184 )
185     s3=0
186     s4=0
187     s5=0
188     s6=0
189     else:
190         x1,y1=ik*h+(h/10),(jk-1)*h+(h/10)
191         x2,y2=ik*h-(h/10),jk*h-(h/10)
192         x3,y3=(ik-1)*h+(h/10),jk*h+(h/10)
193         x4,y4=ik*h-(h/10),(jk+1)*h-(h/10)
194         x5,y5=ik*h+(h/10),jk*h+(h/10)
195         x6,y6=(ik+1)*h-(h/10),jk*h-(h/10)
196         s1=norm_2(proj2Hat(x1, y1, k, N))
197         s2=norm_2(proj2Hat(x2, y2, k, N))
198         s3=norm_2(proj2Hat(x3, y3, k, N))
199         s4=norm_2(proj2Hat(x4, y4, k, N))
200         s5=norm_2(proj2Hat(x5, y5, k, N))
201         s6=norm_2(proj2Hat(x6, y6, k, N))
202         sum1=s1+s2+s3+s4+s5+s6
203     sum1=s1+s2+s3+s4+s5+s6

```

```

202     return sum1
203 ## A set of multiples of $N$ and integer that can be written as
    $iN+1$
204 def Mult(N):
205     mult1=np.array([N+1])
206     mult2=np.array([N])
207     for i in range(2,N):
208         mult1=np.concatenate((mult1, np.array([i*N+1])),axis=0)
209         mult2=np.concatenate((mult2, np.array([i*N])),axis=0)
210     return mult1,mult2
211 def proj1Mult(N):
212     return Mult(N)[0]
213 def proj2Mult(N):
214     return Mult(N)[1]
215 ## Evaluating the component k of the solution
216 def LocalSol(coef,f,x,y,k,N):
217     z=coef[k-1]*f(x,y,k,N)
218     return z
219 #The code initializes a numerical grid and defines a Position
    function
220 #compute indices for a sparse matrix, crucial in solving grid-
    based problems
221 ##The indices varies with grid size and point location,
    indicating a complex grid
222 #structure or boundary conditions
223 x0=0
224 xfin=1
225 N=4
226 h0=1./(N+1)
227 dim0=N**2
228 if N==1:
229     dim=1
230 elif N==2:
231     dim=14
232 else:
233     dim=6+20*(N-2)+8+7*((N-2)**2)
234 I=np.zeros(dim)
235 J=np.zeros(dim)
236 S=np.zeros(dim)
237 B=np.zeros(dim0)
238 def Position(k,N):
239     if N==1:
240         positions=np.array([1])

```

```

241 elif N==2:
242     if k==1:
243         positions=np.array([1,2,3])
244     elif k==2:
245         positions=np.array([4,5,6,7])
246     elif k==3:
247         positions=np.array([8,9,10,11])
248     else:
249         positions=np.array([12,13,14])
250 else:
251     if k==1:
252         positions=np.array([1,2,3])
253         return positions
254     elif k==2:
255         positions=np.array([4,5,6,7,8])
256     elif (k>2) and (k<=N-1):
257         ik=(k-2)*5+4
258         positions=np.array([ik])
259         for i in range(1,5):
260             positions=np.concatenate((positions,np.array([
261 ik+i]))),axis=0)
262     elif k==N:
263         ik=Position(k-1,N)[4]+1
264         positions=np.array([ik])
265         for i in range(1,4):
266             positions=np.concatenate((positions,np.array([
267 ik+i]))),axis=0)
268     elif (k in proj1Mult(N)):
269         check=int((k-1)/N)
270         if check==1:
271             ik=Position(k-1, N)[3]+1
272         else:
273             ik=Position(k-1, N)[4]+1
274         positions=np.array([ik])
275         if check==N-1:
276             for i in range(1,4):
277                 positions=np.concatenate((positions,np.
278 array([ik+i])),axis=0)
279         else:
280             for i in range(1,5):
281                 positions=np.concatenate((positions,np.
282 array([ik+i])),axis=0)
283     elif (k in proj2Mult(N)) and (k>N) and k<N**2:

```

```

280         ik=Position(k-1, N)[6]+1
281         positions=np.array([ik])
282         for i in range(1,5):
283             positions=np.concatenate((positions,np.array([
284             ik+i])),axis=0)
285         elif k==N**2:
286             ik=Position(k-1, N)[4]+1
287             positions=np.array([ik])
288             for i in range(1,3):
289                 positions=np.concatenate((positions,np.array([
290                 ik+i])),axis=0)
291             elif k>(N**2-N+1) and k<N**2:
292                 if k==N**2-N+2:
293                     ik=Position(k-1, N)[3]+1
294                 else:
295                     ik=Position(k-1, N)[4]+1
296                     positions=np.array([ik])
297                     for i in range(1,5):
298                         positions=np.concatenate((positions,np.array([
299                         ik+i])),axis=0)
300                     elif k-1 in proj1Mult(N):
301                         ik=Position(k-1, N)[4]+1
302                         positions=np.array([ik])
303                         for i in range(1,7):
304                             positions=np.concatenate((positions,np.array([
305                             ik+i])),axis=0)
306                     else:
307                         ik=Position(k-1, N)[6]+1
308                         positions=np.array([ik])
309                         for i in range(1,7):
310                             positions=np.concatenate((positions,np.array([
311                             ik+i])),axis=0)
312                 return positions
313
314 for k in range(1,N**2+1):
315     pos=Position(k,N)
316     for i in pos:
317         I[i-1]=k-1
318
319 if N==1:
320     J[0]=0
321 elif N>=2:
322     J[0],J[1],J[2]=0,1,N
323     J[dim-3],J[dim-2],J[dim-1]=dim0-N-1,dim0-2,dim0-1

```

```

318     j=dim0-N+1
319     pos=Position(j, N)[0]
320     pos=int(pos)
321     J[pos-1],J[pos],J[pos+1],J[pos+2]=j-1-N,j-N,j-1,j
322     pos2=Position(N, N)[0]
323     pos2=int(pos2)
324     J[pos2-1],J[pos2],J[pos2+1],J[pos2+2]=N-2,N-1,2*N-2,2*N-1
325     if N==2:
326         pass
327     else:
328         for k in range(1,dim0+1):
329             pos3=Position(k,N)
330             if len(pos3)!=3 or len(pos3)!=4:
331                 pos30=pos3[0]
332                 pos30=int(pos30)
333                 if len(pos3)==5:
334                     if k>1 and k<N:
335                         J[pos30-1],J[pos30],J[pos30+1],J[pos30
+2],J[pos30+3]=k-2,k-1,k,k+N-2,k+N-1
336                     if k>dim0-N+1 and k<dim0:
337                         J[pos30-1],J[pos30],J[pos30+1],J[pos30
+2],J[pos30+3]=k-N-1,k-N,k-2,k-1,k
338                     if k in proj2Mult(N) and k>N and k<dim0:
339                         J[pos30-1],J[pos30],J[pos30+1],J[pos30
+2],J[pos30+3]=k-N-1,k-2,k-1,k+N-2,k+N-1
340                     if k in proj1Mult(N) and k<j:
341                         J[pos30-1],J[pos30],J[pos30+1],J[pos30
+2],J[pos30+3]=k-N-1,k-N,k-1,k,k+N-1
342                     if len(pos3)==7:
343                         J[pos30-1],J[pos30],J[pos30+1],J[pos30+2],J
[pos30+3]=k-N-1,k-N,k-2,k-1,k
344                         J[pos30+4],J[pos30+5]=k+N-2,k+N-1
345 for i in range(dim):
346     i1=int(I[i])+1
347     i2=int(J[i])+1
348     if i1==i2:
349         S[i]=((h0**2)/2)+((h0**2)/2)*gradcomp(i1,i2,N)
350     else:
351         S[i]=((h0**2)/12)+((h0**2)/2)*gradcomp(i1,i2,N)
352 A=csr_matrix((S, (I, J)),shape = (dim0, dim0)).toarray()
353 for i in range(dim0):
354     B[i]=h0**2
355 U_sol=scp.solve(A, B)

```

```

356 def Solution0(x,y):
357     Summ=0
358     for k in range(1,dim0+1):
359         Summ=Summ+LocalSol(U_sol,proj1Hat,x,y,k,N)
360     return Summ
361 Sol=np.vectorize(HatFunck)
362 Solution=np.vectorize(Solution0)
363 x = np.linspace(0, 1, 100)
364 y = np.linspace(0, 1, 100)
365 X0,Y0=np.meshgrid(x, y)
366 Z0=Solution(X0, Y0)
367 im = imshow(Z0,cmap=cm.RdBu,extent=[0, 1, 0, 1], origin='lower',
368             ) # drawing the function
369 colorbar(im)# adding the colobar on the righth
370 # latex fashion title
371 plt.xlabel('X-axis Label')
372 plt.ylabel('Y-axis Label')
373 title('Solution of  $u-\Delta u=1$  for  $N=4$ ')
374 plt.savefig('SolutionPlotSquare1.png')
375 plt.show()

```

Algorithm 1: Manual Python code for the unit square

```

1  # import matplotlib.pyplot as plt
2  import matplotlib.patches as patches
3  import numpy as np
4  import sympy as sp
5  import scipy.linalg as scp
6  from scipy.sparse import csr_matrix
7  ## Transformation to standard triangle
8  # Function to produce the affine transformation from a given
9  # triangle  $P_i(x_i,y_i)$ ,  $i=1,2,3$ ,
10 # the standard triangle  $A(0,0)$ ,  $B(1,0)$ ,  $C(0,1)$ .
11 def Btransf(p1,p2,p3):
12     A=np.zeros((6,6))
13     Lef=np.array([0,0,1,0,0,1])
14     A[:,2]=np.array([1,0,1,0,1,0])
15     A[:,5]=np.array([0,1,0,1,0,1])
16     A[:,0]=np.array([p1[0],0,p2[0],0,p3[0],0])
17     A[:,1]=np.array([p1[1],0,p2[1],0,p3[1],0])
18     A[:,3]=np.array([0,p1[0],0,p2[0],0,p3[0]])
19     A[:,4]=np.array([0,p1[1],0,p2[1],0,p3[1]])
20     B_standard=scp.solve(A, Lef)
21     # The matrix or linear part

```

```

21     B_tr=np.array([[B_standard[0],B_standard[1]], [B_standard
22     [3],B_standard[4]]])
23     # The intercepts
24     vect=np.array([B_standard[2],B_standard[5]])
25     return B_tr,vect
26 # Collecting the matrix
27 def proj1Btransf(p1,p2,p3):
28     return Btransf(p1,p2,p3)[0]
29 # Collecting the intercept
30 def proj2Btransf(p1,p2,p3):
31     return Btransf(p1,p2,p3)[1]
32 # Bijection from  $\{1,2,\dots,N^2\}$  to  $\{(i,j):1\leq i,j\leq N\}$ 
33 # This function helps in this code to locate the hat function
34 # for the grid points
35 # so that one can know the nodes to use and the elements to
36 # consider.
37 def i_k(k,N):
38     ik=min([i for i in range(1,N+1) if k<=i*N])
39     jk=k-(ik-1)*N
40     ik=ik-1
41     jk=jk-1
42     return np.array([ik,jk])
43 ## The standard hat function
44 def standHat(x,y,i):
45     if i==1:
46         z=1-x-y
47     elif i==2:
48         z=x
49     else:
50         z=y
51     return z
52 ## The hat function built upon nodes of the grid points
53 def HatFunck(x,y,k,N):
54     # x,y are point coordinates
55     # k is the hat function's number
56     # the numbers of internal nodes considered
57     h=1./(N+1) # To get N internal nodes
58     ind1=i_k(k,N)
59     ik=ind1[0]+1
60     jk=ind1[1]+1
61     if ik>= (N+1)/2 and jk>= (N+1)/2:
62         z=0
63     grad=np.zeros(2)

```

```

61     else:
62         # $(i_k,i_k)$ represent the position of $\mathbf{v}_{fiz\_k}$ in the
        grid points
63         ik0,jk0=ik, jk-1
64         ik2,jk2=ik, jk+1
65         ik3,jk3=ik-1,jk
66         ik4,jk4=ik+1,jk
67         if x==ik*h and y==jk*h:
68             z=1
69             grad=np.zeros(2)
70         elif x>ik3*h and x<=ik*h:
71             if y>-x+(ik0+jk0)*h and y<=jk*h:
72                 P1=np.array([ik*h,jk*h])
73                 P2=np.array([ik0*h,jk0*h])
74                 P3=np.array([ik3*h,jk3*h])
75                 transf=Btransf(P2, P3, P1)
76                 transfM=proj1Btransf(P2, P3, P1)
77                 transfV=proj2Btransf(P2, P3, P1)
78                 vec=np.array([x,y])
79                 Xtra=np.dot(transfM,vec)+transfV
80                 z=standHat(Xtra[0], Xtra[1],3)
81                 grad=transfM[1]
82             elif y<=-x+(ik+jk)*h and y>jk*h:
83                 P1=np.array([ik*h,jk*h])
84                 P2=np.array([ik3*h,jk3*h])
85                 P3=np.array([ik3*h,jk2*h])
86                 transfM=proj1Btransf(P2, P1, P3)
87                 transfV=proj2Btransf(P2, P1, P3)
88                 vec=np.array([x,y])
89                 Xtra=np.dot(transfM,vec)+transfV
90                 z=standHat(Xtra[0], Xtra[1],2)
91                 grad=transfM[0]
92             elif y>-x+(ik+jk)*h and y<jk2*h:
93                 P1=np.array([ik*h,jk*h])
94                 P2=np.array([ik*h,jk2*h])
95                 P3=np.array([ik3*h,jk2*h])
96                 transfM=proj1Btransf(P1, P3, P2)
97                 transfV=proj2Btransf(P1, P3, P2)
98                 vec=np.array([x,y])
99                 Xtra=np.dot(transfM,vec)+transfV
100                 z=standHat(Xtra[0], Xtra[1],1)
101                 grad=-transfM[0]-transfM[1]
102         else:

```

```

103         z=0
104         grad=np.zeros(2)
105     elif x>ik*h and x<ik4*h:
106         if y<-x+(ik4+jk4)*h and y>=jk*h:
107             P1=np.array([ik*h,jk*h])
108             P2=np.array([ik2*h,jk2*h])
109             P3=np.array([ik4*h,jk4*h])
110             transf=Btransf(P2, P3, P1)
111             transfM=proj1Btransf(P1, P3, P2)
112             transfV=proj2Btransf(P1, P3, P2)
113             vec=np.array([x,y])
114             Xtra=np.dot(transfM,vec)+transfV
115             z=standHat(Xtra[0], Xtra[1],1)
116             grad=-transfM[1]-transfM[0]
117         elif y>=-x+(ik4+jk0)*h and y<jk*h:
118             P1=np.array([ik*h,jk*h])
119             P2=np.array([ik4*h,jk0*h])
120             P3=np.array([ik4*h,jk4*h])
121             transfM=proj1Btransf(P2, P1, P3)
122             transfV=proj2Btransf(P2, P1, P3)
123             vec=np.array([x,y])
124             Xtra=np.dot(transfM,vec)+transfV
125             z=standHat(Xtra[0], Xtra[1],2)
126             grad=transfM[0]
127         elif y<-x+(ik4+jk0)*h and y>jk0*h:
128             P1=np.array([ik*h,jk*h])
129             P2=np.array([ik4*h,jk0*h])
130             P3=np.array([ik0*h,jk0*h])
131             transfM=proj1Btransf(P3, P2, P1)
132             transfV=proj2Btransf(P3, P2, P1)
133             vec=np.array([x,y])
134             Xtra=np.dot(transfM,vec)+transfV
135             z=standHat(Xtra[0], Xtra[1],3)
136             grad=transfM[1]
137         else:
138             z=0
139             grad=np.zeros(2)
140     else:
141         z=0
142         grad=np.array([0,0])
143     return z,grad # we also return the gradient of the  $\psi_k$ 
144 def proj1Hat(x,y,k,N): # To get the hut function
145     return HatFunck(x, y, k, N)[0]

```

```

146 def proj2Hat(x,y,k,N): # To get the gradient of the hat
    function
147     return HatFunck(x, y, k, N)[1]
148 ## Compute the norm squared
149 def norm_2(a):
150     return np.linalg.norm(a)**2
151 ## Computation of the scalar product of the gradient of hat
    function
152 #  $\nabla v_{fi,k} \cdot \nabla v_{fi,l}$  to used in  $\int_0^1 \nabla v_{fi,k} \cdot \nabla v_{fi,l}$ 
153 def gradcomp(k,l,N):
154     h=1./(N+1)
155     ind=i_k(k,N)
156     ik=ind[0]+1
157     jk=ind[1]+1
158     # ind1=i_k(l,N)
159     # il=ind1[0]+1
160     # jl=ind1[1]+1
161     if ik>=(N+1)/2 and jk>=(N+1)/2:
162         sum1=0
163     else:
164         # building points for computing successfully the gradient
165         if l!=k:
166             if l==k-1:
167                 x1,y1=ik*h+(h/10),(jk-1)*h+(h/10)
168                 x2,y2=ik*h-(h/10),jk*h-(h/10)
169             elif l==k-N:
170                 x1,y1=ik*h-(h/10),jk*h-(h/10)
171                 x2,y2=(ik-1)*h+(h/10),jk*h+(h/10)
172             elif l==k-N+1:
173                 x1,y1=(ik-1)*h+(h/10),jk*h+(h/10)
174                 x2,y2=ik*h-(h/10),(jk+1)*h-(h/10)
175             elif l==k+1:
176                 x1,y1=ik*h-(h/10),(jk+1)*h-(h/10)
177                 x2,y2=ik*h+(h/10),jk*h+(h/10)
178             elif l==k+N:
179                 x1,y1=ik*h+(h/10),jk*h+(h/10)
180                 x2,y2=(ik+1)*h-(h/10),jk*h-(h/10)
181             elif l==k+N-1:
182                 x1,y1=(ik+1)*h-(h/10),jk*h-(h/10)
183                 x2,y2=ik*h+(h/10),(jk-1)*h+(h/10)
184             else:
185                 x1,y1=(ik+2)*h,(jk+2)*h

```

```

186         x2,y2=(ik-2)*h,(jk-2)*h
187         s1=np.dot(proj2Hat(x1, y1, k, N),proj2Hat(x1, y1, 1
, N))
188         s2=np.dot(proj2Hat(x2, y2, k, N),proj2Hat(x2, y2, 1
, N))
189         s3=0
190         s4=0
191         s5=0
192         s6=0
193     else:
194         x1,y1=ik*h+(h/10),(jk-1)*h+(h/10)
195         x2,y2=ik*h-(h/10),jk*h-(h/10)
196         x3,y3=(ik-1)*h+(h/10),jk*h+(h/10)
197         x4,y4=ik*h-(h/10),(jk+1)*h-(h/10)
198         x5,y5=ik*h+(h/10),jk*h+(h/10)
199         x6,y6=(ik+1)*h-(h/10),jk*h-(h/10)
200         s1=norm_2(proj2Hat(x1, y1, k, N))
201         s2=norm_2(proj2Hat(x2, y2, k, N))
202         s3=norm_2(proj2Hat(x3, y3, k, N))
203         s4=norm_2(proj2Hat(x4, y4, k, N))
204         s5=norm_2(proj2Hat(x5, y5, k, N))
205         s6=norm_2(proj2Hat(x6, y6, k, N))
206         sum1=s1+s2+s3+s4+s5+s6
207     sum1=s1+s2+s3+s4+s5+s6
208     return sum1
209 ## A set of multiples of $N$ and integer that can be written as
    $iN+1$
210 def Mult(N):
211     mult1=np.array([N+1])
212     mult2=np.array([N])
213     for i in range(2,N):
214         mult1=np.concatenate((mult1, np.array([i*N+1])),axis=0)
215         mult2=np.concatenate((mult2, np.array([i*N])),axis=0)
216     return mult1,mult2
217 def proj1Mult(N):
218     return Mult(N)[0]
219 def proj2Mult(N):
220     return Mult(N)[1]
221 ## Evaluating the component k of the solution
222 def LocalSol(coef,f,x,y,k,N):
223     z=coef[k-1]*f(x,y,k,N)
224     return z
225

```

```

226 from mpl_toolkits.mplot3d import Axes3D
227 from matplotlib import cm
228 from matplotlib.ticker import LinearLocator
229 from pylab import meshgrid,cm,imshow,contour,clabel,colorbar,
    axis,title,show
230 x0=0
231 xfin=1
232 N=16
233 h0=1./(N+1)
234 dim0=N**2
235 if N==1:
236     dim=1
237 elif N==2:
238     dim=14
239 else:
240     dim=6+20*(N-2)+8+7*((N-2)**2)
241 I=np.zeros(dim)
242 J=np.zeros(dim)
243 S=np.zeros(dim)
244 B=np.zeros(dim0)
245 def Position(k,N):
246     if N==1:
247         positions=np.array([1])
248     elif N==2:
249         if k==1:
250             positions=np.array([1,2,3])
251         elif k==2:
252             positions=np.array([4,5,6,7])
253         elif k==3:
254             positions=np.array([8,9,10,11])
255         else:
256             positions=np.array([12,13,14])
257     else:
258         if k==1:
259             positions=np.array([1,2,3])
260             return positions
261         elif k==2:
262             positions=np.array([4,5,6,7,8])
263         elif (k>2) and (k<=N-1):
264             ik=(k-2)*5+4
265             positions=np.array([ik])
266             for i in range(1,5):
267                 positions=np.concatenate((positions,np.array([

```

```

    ik+i]))),axis=0)
268     elif k==N:
269         ik=Position(k-1,N)[4]+1
270         positions=np.array([ik])
271         for i in range(1,4):
272             positions=np.concatenate((positions,np.array([
    ik+i]))),axis=0)
273     elif (k in proj1Mult(N)):
274         check=int((k-1)/N)
275         if check==1:
276             ik=Position(k-1, N)[3]+1
277         else:
278             ik=Position(k-1, N)[4]+1
279         positions=np.array([ik])
280         if check==N-1:
281             for i in range(1,4):
282                 positions=np.concatenate((positions,np.
    array([ik+i]))),axis=0)
283         else:
284             for i in range(1,5):
285                 positions=np.concatenate((positions,np.
    array([ik+i]))),axis=0)
286     elif (k in proj2Mult(N)) and (k>N) and k<N**2:
287         ik=Position(k-1, N)[6]+1
288         positions=np.array([ik])
289         for i in range(1,5):
290             positions=np.concatenate((positions,np.array([
    ik+i]))),axis=0)
291     elif k==N**2:
292         ik=Position(k-1, N)[4]+1
293         positions=np.array([ik])
294         for i in range(1,3):
295             positions=np.concatenate((positions,np.array([
    ik+i]))),axis=0)
296     elif k>(N**2-N+1) and k<N**2:
297         if k==N**2-N+2:
298             ik=Position(k-1, N)[3]+1
299         else:
300             ik=Position(k-1, N)[4]+1
301             positions=np.array([ik])
302             for i in range(1,5):
303                 positions=np.concatenate((positions,np.array([
    ik+i]))),axis=0)

```

```

304         elif k-1 in proj1Mult(N):
305             ik=Position(k-1, N)[4]+1
306             positions=np.array([ik])
307             for i in range(1,7):
308                 positions=np.concatenate((positions,np.array([
309 ik+i]))),axis=0)
310         else:
311             ik=Position(k-1, N)[6]+1
312             positions=np.array([ik])
313             for i in range(1,7):
314                 positions=np.concatenate((positions,np.array([
315 ik+i]))),axis=0)
316         return positions
317
318 for k in range(1,N**2+1):
319     pos=Position(k,N)
320     for i in pos:
321         I[i-1]=k-1
322
323 if N==1:
324     J[0]=0
325 elif N>=2:
326     J[0],J[1],J[2]=0,1,N
327     J[dim-3],J[dim-2],J[dim-1]=dim0-N-1,dim0-2,dim0-1
328     j=dim0-N+1
329     pos=Position(j, N)[0]
330     pos=int(pos)
331     J[pos-1],J[pos],J[pos+1],J[pos+2]=j-1-N,j-N,j-1,j
332     pos2=Position(N, N)[0]
333     pos2=int(pos2)
334     J[pos2-1],J[pos2],J[pos2+1],J[pos2+2]=N-2,N-1,2*N-2,2*N-1
335     if N==2:
336         pass
337     else:
338         for k in range(1,dim0+1):
339             pos3=Position(k,N)
340             if len(pos3)!=3 or len(pos3)!=4:
341                 pos30=pos3[0]
342                 pos30=int(pos30)
343                 if len(pos3)==5:
344                     if k>1 and k<N:
345                         J[pos30-1],J[pos30],J[pos30+1],J[pos30
+2],J[pos30+3]=k-2,k-1,k,k+N-2,k+N-1
346                     if k>dim0-N+1 and k<dim0:

```

```

344         J[pos30-1], J[pos30], J[pos30+1], J[pos30
+2], J[pos30+3]=k-N-1, k-N, k-2, k-1, k
345         if k in proj2Mult(N) and k>N and k<dim0:
346             J[pos30-1], J[pos30], J[pos30+1], J[pos30
+2], J[pos30+3]=k-N-1, k-2, k-1, k+N-2, k+N-1
347             if k in proj1Mult(N) and k<j:
348                 J[pos30-1], J[pos30], J[pos30+1], J[pos30
+2], J[pos30+3]=k-N-1, k-N, k-1, k, k+N-1
349                 if len(pos3)==7:
350                     J[pos30-1], J[pos30], J[pos30+1], J[pos30+2], J
[pos30+3]=k-N-1, k-N, k-2, k-1, k
351                     J[pos30+4], J[pos30+5]=k+N-2, k+N-1
352 for i in range(dim):
353     i1=int(I[i])+1
354     i2=int(J[i])+1
355     if i1==i2:
356         S[i]=((h0**2)/2)+((h0**2)/2)*gradcomp(i1, i2, N)
357     else:
358         S[i]=((h0**2)/12)+((h0**2)/2)*gradcomp(i1, i2, N)
359 A=csr_matrix((S, (I, J)), shape = (dim0, dim0)).toarray()
360 for i in range(dim0):
361     B[i]=h0**2
362 U_sol=scp.solve(A, B)
363 def Solution0(x,y):
364     Summ=0
365     for k in range(1, dim0+1):
366         Summ=Summ+LocalSol(U_sol, proj1Hat, x, y, k, N)
367     return Summ
368 Sol=np.vectorize(HatFunck)
369 Solution=np.vectorize(Solution0)
370 x = np.linspace(0, 1, 100)
371 y = np.linspace(0, 1, 100)
372 X0,Y0=np.meshgrid(x, y)
373 Z0=Solution(X0, Y0)
374 im = imshow(Z0, cmap=cm.RdBu, extent=[0, 1, 0, 1], origin='lower',
) # drawing the function
375 # adding the Contour lines with labels
376 # cset = contour(Z0, np.arange(-1, 1.5, 0.2), linewidths=2, cmap=cm.
Set2)
377 # clabel(cset, inline=True, fmt='%1.1f', fontsize=10)
378 colorbar(im) # adding the colobar on the righth
379 # latex fashion title
380 plt.xlabel('X-axis Label')

```

```

381 plt.ylabel('Y-axis Label')
382 title('Solution of $u-\Delta u=1$ for $N=2$')
383
384 # Access the current axes, and hide the top and right spines
385 ax = plt.gca()
386 ax.spines['top'].set_color('none')
387 ax.spines['right'].set_color('none')
388
389 # Optional: Move the bottom and left spines outward
390 ax.spines['bottom'].set_position(('outward', 10))
391 ax.spines['left'].set_position(('outward', 10))
392
393 # Optional: Hide the ticks on the top and right edges
394 ax.xaxis.tick_bottom()
395 ax.yaxis.tick_left()
396
397 # Set the background color of the figure and the axes
398 ax.set_facecolor('white') # Set axes background color to white
399 ax.figure.set_facecolor('white') # Set the figure background
    color to white
400
401 # Create a white rectangle in the top right corner
402 # The rectangle starts at (0.55, 0.55) and extends to (1, 1)
403 rect = patches.Rectangle((0.55, 0.55), 0.5, 0.5, linewidth=1,
    edgecolor='none', facecolor='white', zorder=10)
404 ax.add_patch(rect)
405
406 # Ensure the saved figure doesn't have a background color
407 plt.savefig('SolutionPlots.png', bbox_inches='tight', facecolor
    ='white')
408
409 plt.show()

```

Algorithm 2: Manual code for the L-domain

Appendix for FEniCS

```
1 #This is the code for the unit square domain
2
3 from fenics import *
4 import numpy as np
5
6 # Create mesh and define function space
7 mesh = UnitSquareMesh(32, 32)
8 V = FunctionSpace(mesh, 'P', 1)
9
10 # Define boundary condition
11 u_D = Constant(0)
12
13 def boundary(x, on_boundary):
14     return on_boundary
15
16 bc = DirichletBC(V, u_D, boundary)
17
18 # Define variational problem
19 u = TrialFunction(V)
20 v = TestFunction(V)
21 f = Constant(1)
22 a = (u*v + dot(grad(u), grad(v)))*dx
23 L = f*v*dx
24
25 # Compute solution
26 u = Function(V)
27 solve(a == L, u, bc)
28
29 # Plot solution
30 import matplotlib.pyplot as plt
31 plot(u)
32
33 plt.savefig("unit_square_solution")
34
35 plt.show()
36
37 # Compute error in L2 norm
38 error_L2 = errornorm(u_D, u, 'L2')
39
40 # Print errors
41 print('error_L2 =', error_L2)
```

```

42
43 # Compute maximum error at vertices
44 vertex_values_u_D = u_D.compute_vertex_values(mesh)
45 vertex_values_u = u.compute_vertex_values(mesh)
46 error_max = np.max(np.abs(vertex_values_u_D - vertex_values_u))
47
48 # Print maximum error
49 print('error_max =', error_max)

```

Algorithm 3: FEniCS code for the unit square

```

1
2 #This is a code for the error vs mesh size on a unit square
   domain
3
4 from fenics import *
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8 # Define boundary condition
9 u_D = Expression('0', degree=1)
10
11 def boundary(x, on_boundary):
12     return on_boundary
13
14 # Define variational problem
15 def solve_pde(N):
16     mesh = UnitSquareMesh(N, N)
17     V = FunctionSpace(mesh, 'P', 1)
18     bc = DirichletBC(V, u_D, boundary)
19     u = TrialFunction(V)
20     v = TestFunction(V)
21     f = Constant(1)
22     a = (u*v + dot(grad(u), grad(v)))*dx
23     L = f*v*dx
24     u = Function(V)
25     solve(a == L, u, bc)
26     return u, mesh
27
28 # Compute error in L2 norm and record mesh size
29 mesh_sizes = [8, 16, 32, 64, 128]
30 errors_L2 = []
31 h = []
32

```

```

33 for N in mesh_sizes:
34     u, mesh = solve_pde(N)
35     u_exact = interpolate(u_D, V)
36     error_L2 = errornorm(u_exact, u, 'L2')
37     errors_L2.append(error_L2)
38     h.append(mesh.hmax())
39
40 # Plotting the error vs. mesh size
41 plt.loglog(h, errors_L2, '-o')
42 plt.xlabel('Mesh size (h)')
43 plt.ylabel('L2 error')
44 plt.title('L2 error vs. Mesh size')
45 plt.grid(True)
46
47 plt.savefig("error_solution")
48
49 plt.show()

```

Algorithm 4: FEniCS code for error vs mesh size

```

1 #FEniCS code for the L-shaped domain
2
3 from fenics import *
4 import matplotlib.pyplot as plt
5
6 #Creating a mesh for the unit square
7 mesh = UnitSquareMesh(32, 32)
8
9 #Defining a subdomain for the L-shape
10 class LShape(SubDomain):
11     def inside(self, x, on_boundary):
12         return x[0] <= 0.5 or x[1] <= 0.5
13
14 #Marking all cells, 1 for those inside the L-shape, 0 otherwise
15 domains = MeshFunction('size_t', mesh, mesh.topology().dim())
16 domains.set_all(0)
17 lshape = LShape()
18 lshape.mark(domains, 1)
19
20 #Creating a new mesh only with the cells marked as 1
21 lshape_mesh = SubMesh(mesh, domains, 1)
22
23 #Defining function space on the L-shaped mesh
24 V = FunctionSpace(lshape_mesh, 'P', 1)

```

```

25
26 #Defining boundary condition
27 def boundary(x, on_boundary):
28     return on_boundary
29 bc = DirichletBC(V, Constant(0), boundary)
30
31 #Defining variational problem
32 u = TrialFunction(V)
33 v = TestFunction(V)
34 f = Constant(1)
35 a = (u*v + dot(grad(u), grad(v)))*dx
36 L = f*v*dx
37
38 #Computing solution
39 u = Function(V)
40 solve(a == L, u, bc)
41
42 #Saving solution to file (optional)
43 vtkfile = File('lshape_solution.pvd')
44 vtkfile << u
45
46 #Plotting the solution
47 plot(u)
48 plt.title('Solution on L-shaped Domain')
49
50 plt.savefig('lshape_solution.png')
51 plt.show()

```

Algorithm 5: FEniCS code for the L-shaped